



auth.md — Orvio Authentication & Identity Specification

Version: 0.1 (MVP-ready)

Last updated: 2026-09-28

Owner: Engineering (Code X)

Applies to: Frontend, Backend, DevOps, QA, AI coding agents

0. How to Use This Document

This is the **authoritative source of truth** for authentication and identity in Orvio. It defines how users are created, verified, authenticated, sessioned, recovered, and secured.

Chain of truth:

PRD → architecture.md → database.md → api-contract.md → auth.md → Implementation → Tests

Rules:

- Do not change auth behavior without updating this doc and tests.
- Do not bypass authN/authZ boundaries.
- AI agents must read this before implementing auth-related features.

1. Core Principle

Treat authentication as a **security-critical system**.

This document defines:

- Who a user is (identity).
- How accounts are created/verified.
- How credentials are established/stored.
- How users authenticate (login).
- How sessions/tokens are created/maintained/expired/revoked.
- How auth state reaches frontend/backend.
- How failures are handled.
- How recovery works.

Status labels:

- **Confirmed requirement:** From PRD/architecture.
- **Security recommendation:** Engineering guidance.
- **Assumption:** Reasonable inference, may need validation.
- **OPEN DECISION:** Not yet decided; do not assume.

2. Authentication vs Authorization

Authentication (AuthN)

"Who are you?"

- Establishes trusted identity via credentials (email/phone + password).
- Issues tokens/sessions representing that identity.

Authorization (AuthZ)

"What are you allowed to do?"

- Determines permissions based on role, org membership, resource ownership.
- Enforced server-side (see `api-contract.md`, `database.md`).

Interaction:

1. AuthN verifies identity → issues JWT.
2. AuthZ checks JWT claims (`userId`, `orgId`, `role`) + policy before allowing action.

Important: Frontend route guards/hidden UI are **UX only**. Backend must independently enforce AuthZ.

3. Authentication Architecture

3.1 Components

- **Frontend (React):** Login/signup forms, token storage, auth state.
- **Backend (Fastify):** Auth endpoints, JWT issuance/validation, password hashing.
- **Database (PostgreSQL):** `users`, `user_memberships`, `organizations`.
- **Email provider (future):** Verification, reset emails.
- **OAuth providers (future):** Google, Microsoft.
- **MFA provider (future):** TOTP/SMS.
- **Cache (future):** Token revocation list, rate limits.

3.2 Flow Diagram

```
sequenceDiagram
    participant U as User
    participant F as Frontend
    participant A as Auth API
    participant DB as Database
    participant E as Email Provider

    U->>F: Enter credentials
    F->>A: POST /auth/login
    A->>DB: Verify user & hash
    DB-->>A: User record
    A->>A: Generate access + refresh tokens
    A-->>F: Tokens + user profile
    F->>U: Redirect to dashboard

    U->>F: Click "Reset Password"
    F->>A: POST /auth/reset-password
    A->>DB: Validate email
    A->>E: Send reset email
    E-->>U: Email with link
    U->>F: Open link, set new password
    F->>A: POST /auth/reset-password/confirm
    A->>DB: Update hash
    DB-->>A: Success
    A-->>F: OK
    F->>U: Redirect to login
```

4. Identity Model

4.1 User Entity

Table: users (see database.md).

Fields:

- id UUID (primary key)
- email TEXT UNIQUE (primary identifier)
- phone TEXT (optional, E.164)
- password_hash TEXT (argon2/bcrypt)
- status ENUM (active, suspended, deleted)
- mfa_enabled BOOL (future)
- created_at, updated_at, deleted_at

Ownership: User owns their identity; orgs reference via user_memberships.

4.2 Organization Membership

Table: `user_memberships`.

Fields:

- `org_id` UUID (tenant)
- `user_id` UUID (user)
- `role` ENUM (owner, manager, cashier, storekeeper, accountant)
- `status` ENUM (active, inactive, deleted)

Purpose: Links user to org with role for AuthZ.

5. Authentication Methods

5.1 Primary (MVP)

- **Email + Password** (or phone + password; OPEN DECISION).
- **JWT tokens** (access + refresh).

5.2 Future

- **OAuth/Social:** Google, Microsoft.
- **MFA:** TOTP/SMS.
- **Magic link:** Email-based passwordless.

OPEN DECISION: Enable phone login in MVP? (Recommendation: email only for MVP simplicity.)

6. Credential Management

6.1 Passwords

Hashing: `argon2id` (preferred) or `bcrypt` (cost ≥ 12).

Storage: `password_hash` in `users` table; never plaintext.

Policy (MVP):

- **Min length:** 8 chars.
- **Complexity:** OPEN DECISION (recommendation: no mandatory complexity; enforce length).
- **Breach check:** OPEN DECISION (future: HaveIBeenPwned API).

6.2 Password Reset

Flow:

1. User requests reset (email).
2. Backend generates one-time token (1-hour expiry).
3. Email sent with link (/reset-password?token=...).
4. User submits new password.
5. Backend validates token, updates hash, invalidates sessions (future).

Security:

- Rate limit requests (5/hour per email).
- Token single-use, cryptographically random.

7. Session & Token Architecture

7.1 Token Types

- **Access token:** Short-lived (15 min), JWT, in memory.
- **Refresh token:** Longer (7–30 days), JWT or opaque, httpOnly cookie or secure storage.

7.2 Token Claims (Access)

```
{
  "sub": "user_uuid",
  "org_id": "org_uuid",
  "role": "owner",
  "iat": 1727560000,
  "exp": 1727560900
}
```

7.3 Token Issuance

On login:

1. Validate credentials.
2. Generate access + refresh tokens.
3. Return to client (access in body, refresh in httpOnly cookie).

On refresh:

1. Validate refresh token.
2. Issue new access + refresh (rotation).
3. Revoke old refresh (future: store in DB/cache).

7.4 Storage

- **Access token:** Memory (React state/Zustand); never localStorage.
- **Refresh token:** httpOnly cookie (preferred) or secure storage.

Security recommendation: httpOnly cookies to prevent XSS theft.

8. Authentication Endpoints

See `api-contract.md` for full contract. Summary:

ID	Method	Endpoint	Purpose	Auth
AUTH-001	POST	/auth/register	Register user	No
AUTH-002	POST	/auth/login	Login	No
AUTH-003	POST	/auth/refresh	Refresh token	No
AUTH-004	POST	/auth/logout	Logout	Yes
AUTH-005	POST	/auth/reset-password	Request reset	No
AUTH-006	POST	/auth/reset-password/confirm	Confirm reset	No
AUTH-007	GET	/auth/me	Get current user	Yes

9. Frontend Authentication Behavior

9.1 Auth State

- **Store:** Zustand or Context (authStore).
- **Fields:** user, orgId, role, isAuthenticated.
- **Persistence:** Refresh token in httpOnly cookie; access token in memory.

9.2 Route Guards

- **Protected routes:** Check isAuthenticated; redirect to /login if false.
- **Role guards:** Check role for UX (backend still enforces).

9.3 Token Handling

- **Attach:** Authorization: Bearer <access_token> to API requests.
- **Refresh:** On 401, call /auth/refresh; retry original request.
- **Logout:** Clear auth state, call /auth/logout.

9.4 Loading/Error States

- **Loading:** Show spinner during login/register/reset.
- **Error:** Display user-friendly message; log correlation ID.

10. Backend Authentication Behavior

10.1 Middleware

- **Auth middleware:** Validates JWT, extracts `userId`, `orgId`, `role`.
- **Org context:** Ensures `org_id` matches membership.

10.2 Validation

- **Input:** Zod schemas for login/register/reset.
- **Rate limiting:** On login, reset, register.

10.3 Errors

- 401 `INVALID_CREDENTIALS` (login).
- 401 `SESSION_EXPIRED` (token expired).
- 403 `FORBIDDEN` (authenticated, no permission).

11. Email Verification

Status: OPEN DECISION (defer to Phase 2 for MVP speed).

If implemented:

- Send verification email on register.
- Token-based link (`/verify-email?token=...`).
- Mark `email_verified_at` on confirm.
- Restrict actions until verified (optional).

12. Multi-Factor Authentication (MFA)

Status: OPEN DECISION (Phase 2+).

If implemented:

- TOTP (Google Authenticator) or SMS.
- Enforce for Owner roles (optional).
- Backup codes.

13. OAuth / Social Login

Status: OPEN DECISION (Phase 2+).

If implemented:

- Google, Microsoft.
- Link OAuth account to existing user or create new.
- Merge identities carefully (prevent account takeover).

14. Device / Session Management

MVP: Single session per user (simplified).

Future:

- List active sessions (device, IP, last active).
- Revoke individual sessions.
- Force logout everywhere.

15. Account Recovery

15.1 Password Reset

See §6.2.

15.2 Account Takeover Protection

- Email confirmation for sensitive changes (future).
- Audit log of password resets.

15.3 Suspended/Deleted Accounts

- **Suspended:** Login fails with 403 ACCOUNT_SUSPENDED.
- **Deleted:** Soft delete; prevent login.

16. Account Lifecycle

State	Description	Login Allowed	Actions
pending	Registered, not verified (if email verification enabled)	No (or limited)	Verify email
active	Verified, good standing	Yes	All permitted
suspended	Admin action (abuse, non-payment)	No	Contact support
deleted	User-requested or admin	No	Data retained per policy

17. Security Considerations

17.1 OWASP Top 10

- **A01 Broken Access Control:** Enforce AuthZ server-side.
- **A02 Cryptographic Failures:** Use argon2/bcrypt, HTTPS.
- **A03 Injection:** Parameterized queries (Prisma).
- **A07 Auth Failures:** Rate limit, strong passwords, token rotation.

17.2 Token Security

- **Short-lived access:** 15 min.
- **Refresh rotation:** New refresh on each use.
- **Revocation:** Future (store in DB/cache).

17.3 Brute-Force Protection

- Rate limit login (5/min per IP/email).
- Lock account after N failures (OPEN DECISION).

17.4 XSS/CSRF

- **XSS:** httpOnly cookies, CSP headers.
- **CSRF:** Not applicable if JWT in Authorization header; if cookies used, CSRF tokens required.

18. Database Schema (Auth-Related)

See `database.md` for full schema. Summary:

- `users`: Identity, password hash, status.
- `user_memberships`: Org-role linkage.
- `organizations`: Tenant boundary.
- `audit_logs`: Auth events (login, reset, etc.).

19. API Contract (Auth Endpoints)

See `api-contract.md` for full contract. Key examples:

POST /auth/login

Request:

```
{
  "email": "user@example.com",
  "password": "secret"
}
```

Success (200):

```
{
  "data": {
    "accessToken": "...",
    "refreshToken": "...",
    "user": { "id": "u1", "email": "user@example.com" }
  },
  "meta": { "requestId": "req_123" }
}
```

Errors: 401 INVALID_CREDENTIALS, 400 VALIDATION_ERROR.

POST /auth/refresh

Request: Refresh token (httpOnly cookie).

Success (200): New access + refresh tokens.

Errors: 401 SESSION_EXPIRED.

20. Testing Requirements

- **Unit tests:** Password hashing, token generation/validation.
- **Integration tests:** Login, refresh, reset flows.
- **Security tests:** Brute-force rate limit, token expiry, revoked tokens.
- **E2E tests:** Full login → protected route → logout.

21. Observability

- **Logs:** Auth events (login success/failure, reset requested/completed).
- **Metrics:** Login success rate, reset rate, token refresh rate.
- **Alerts:** Spike in login failures (brute-force).

Never log: Passwords, tokens, secrets.

22. Open Decisions

ID	Decision	Options	Impact	Recommended Info Needed
AUTH-01	Email verification	Required vs optional	UX vs security	Threat model, user base
AUTH-02	MFA	TOTP vs SMS vs none	Security vs complexity	User needs, compliance
AUTH-03	OAuth providers	Google, Microsoft, none	UX vs integration effort	User preference
AUTH-04	Session management	Single vs multi-session	UX vs complexity	User workflows
AUTH-05	Password policy	Length only vs complexity	Security vs UX	Industry standards

23. AI Coding Agent Rules

AI agents must:

- Read this doc before changing auth code.
- Never bypass auth middleware or token validation.
- Use existing token generation/validation utilities.
- Add tests for any auth change.
- Never log sensitive data (passwords, tokens).
- Preserve backward compatibility in auth flows.

24. Authentication Quality Checklist

- ☐ Passwords hashed with argon2/bcrypt.
- ☐ Tokens short-lived, rotated.
- ☐ httpOnly cookies for refresh.
- ☐ Rate limiting on login/reset.
- ☐ AuthZ enforced server-side.
- ☐ Audit logs for auth events.
- ☐ Tests for all auth flows.
- ☐ No sensitive data in logs.
- ☐ Tenant isolation in auth context.
- ☐ Error messages user-friendly, not leaky.

End of auth.md